MATH3881/5881: Statistical Machine Learning Theory

Week 7: Tutorial Solutions

Sahani Pathiraja *UNSW, Sydney*

Exercise 1.

Solutions for all exercises within lecture notes not discussed in class:

Exercise 7.4.2 (Lecture Notes)

Proof: Using the total derivative formula,

$$\frac{dh^{(t)}}{dW_{ij}} = \frac{\partial h^{(t)}}{\partial W_{ij}} + \frac{\partial h^{(t)}}{\partial h^{(t-1)}} \frac{dh^{(t-1)}}{dW_{ij}}$$

where using chain rule, $\frac{\partial h^{(t)}}{\partial W_{ij}} = \dot{\phi}(a^{(t)}) h_j^{(t-1)}$. Notice this leads to a recursive formula, so then by similar arguments,

$$\frac{dh^{(t-1)}}{dW_{ij}} = \frac{\partial h^{(t-1)}}{\partial W_{ij}} + \frac{dh^{(t-2)}}{dW_{ij}} \frac{\partial h^{(t-1)}}{\partial h^{(t-2)}}$$

and substituting back into the previous expression yields

$$\frac{dh^{(t)}}{dW_{ij}} = \frac{\partial h^{(t)}}{\partial W_{ij}} + \left(\frac{\partial h^{(t-1)}}{\partial W_{ij}} + \frac{\partial h^{(t-1)}}{\partial h^{(t-2)}} \frac{dh^{(t-2)}}{dW_{ij}} \right) \frac{\partial h^{(t)}}{\partial h^{(t-1)}} = \frac{dh^{(t-2)}}{dW_{ij}} \underbrace{\frac{\partial h^{(t)}}{\partial h^{(t-1)}} \frac{\partial h^{(t-1)}}{\partial h^{(t-2)}}}_{\frac{\partial h^{(t)}}{\partial h^{(t-2)}}} + \sum_{k=0}^{1} \frac{\partial h^{(t-k)}}{\partial W_{ij}} \frac{\partial h^{(t)}}{\partial h^{(t-k)}}$$

then consider derivative at $t-2$ by the same formula,

$$\frac{dh^{(t-2)}}{dW_{ij}} = \frac{\partial h^{(t-2)}}{\partial W_{ij}} + \frac{dh^{(t-3)}}{dW_{ij}} \frac{\partial h^{(t-2)}}{\partial h^{(t-3)}}$$

and substituting back into our expression for $\frac{dh^{(t)}}{dW_{ij}}$ yields

$$\begin{aligned} \frac{dh^{(t)}}{dW_{ij}} &= \left(\frac{\partial h^{(t-2)}}{\partial W_{ij}} + \frac{dh^{(t-3)}}{dW_{ij}} \frac{\partial h^{(t-2)}}{\partial h^{(t-3)}} \right) \frac{\partial h^{(t)}}{\partial h^{(t-2)}} + \sum_{k=0}^{1} \frac{\partial h^{(t-k)}}{\partial W_{ij}} \frac{\partial h^{(t)}}{\partial h^{(t-k)}} \\ &= \frac{dh^{(t-3)}}{dW_{ij}} \frac{\partial h^{(t)}}{\partial h^{(t-3)}} + \sum_{k=0}^{2} \frac{\partial h^{(t-k)}}{\partial W_{ij}} \frac{\partial h^{(t)}}{\partial h^{(t-k)}} \end{aligned}$$

proceeding inductively, we have that

$$\begin{aligned}\frac{dh^{(t)}}{dW_{ij}} &= \frac{dh^{(t-t)}}{dW_{ij}}\frac{\partial h^{(t)}}{\partial h^{(t-t)}} + \sum_{k=0}^{t-1}\frac{\partial h^{(t-k)}}{\partial W_{ij}}\frac{\partial h^{(t)}}{\partial h^{(t-k)}} \\ &= \sum_{k=0}^{t-1}\frac{\partial h^{(t-k)}}{\partial W_{ij}}\frac{\partial h^{(t)}}{\partial h^{(t-k)}}\end{aligned}$$

with $\frac{dh^{(0)}}{dW_{ij}} = 0$. This is equivalent to the provided expression with a change of index. ■

Exercise 7.4.4 (Lecture Notes)

Proof: We first consider the derivative with respect to U_{ij}. By the chain rule,

$$\frac{d\mathcal{L}}{dU_{ij}} = \sum_{k=1}^{T}\left(\frac{\partial h^{(k)}}{\partial U_{ij}}\right)^{\top}\Delta_h^{(k)} \tag{7.1}$$

where $\Delta_h^{(k)} \in \mathbb{R}^{m\times 1}$ is the total error signal at time k. Using Assumption **??**, we have

$$\frac{\partial h^{(k)}}{\partial U_{ij}} = \dot{\phi}(a_i^{(k)})\, x_j^{(k)}\, e_i \in \mathbb{R}^{m\times 1}, \tag{7.2}$$

where e_i is the ith standard basis vector in $\mathbb{R}^m$. Therefore,

$$\frac{d\mathcal{L}}{dU_{ij}} = \sum_{k=1}^{T}\dot{\phi}(a_i^{(k)})\, x_j^{(k)}\, (\Delta_h^{(k)})_i, \qquad i = 1,\dots,m, \quad j = 1,\dots,d). \tag{7.3}$$

Equivalently, in matrix form,

$$\nabla_U \mathcal{L} = \sum_{k=1}^{T}\left(\Delta_h^{(k)} \odot \dot{\phi}(a^{(k)})\right)\left(x^{(k)}\right)^{\top}, \tag{7.4}$$

where $\odot$ denotes the Hadamard product. Here $\Delta_h^{(k)} \odot \dot{\phi}(a^{(k)}) \in \mathbb{R}^{m\times 1}$, so the resulting gradient has the same shape as U.

Now consider the bias vector b. Since each component of b is added directly to the corresponding pre-activation coordinate, we obtain

$$\frac{\partial h^{(k)}}{\partial b_j} = \dot{\phi}(a_j^{(k)})\, e_j, \qquad j = 1,\dots,m, \tag{7.5}$$

and hence

$$\frac{d\mathcal{L}}{db_j} = \sum_{k=1}^{T}\dot{\phi}(a_j^{(k)})\, (\Delta_h^{(k)})_j. \tag{7.6}$$

In vector form,

$$\nabla_b \mathcal{L} = \sum_{k=1}^{T} \Delta_h^{(k)} \odot \dot{\phi}(a^{(k)}), \tag{7.7}$$

which has dimension $m \times 1$. Recall that in practice, one often defines the local pre-activation error

$$\Delta_a^{(k)} := \Delta_h^{(k)} \odot \dot{\phi}(a^{(k)}), \tag{7.8}$$

so that the gradients take the compact form

$$\nabla_U \mathcal{L} = \sum_{k=1}^{T} \Delta_a^{(k)} \big(x^{(k)}\big)^\top, \qquad \nabla_b \mathcal{L} = \sum_{k=1}^{T} \Delta_a^{(k)}. \tag{7.9}$$

This is the form that is typically implemented in Backpropagation Through Time: the error signal is propagated backward through time via $\Delta_h^{(k)}$, converted into the pre-activation error $\delta_a^{(k)}$ by multiplying elementwise with the activation derivative, and then accumulated into the parameter gradients. ■

Exercise 7.8.1 (Lecture Notes)

Proof: Firstly,

$$\Delta_{\tilde{h}}^{(k)} = \delta_{\tilde{h}}^{(k)} + \sum_{l=k+1}^{T} \left(\frac{\partial \tilde{h}^{(l)}}{\partial \tilde{h}^{(k)}} \right) \delta_{\tilde{h}}^{(l)} \tag{7.10}$$

Now for $l > k$, the chain rule implies

$$\begin{aligned} \Delta_{\tilde{h}}^{(k)} &= \delta_{\tilde{h}}^{(k)} + \left(\frac{\partial \tilde{h}^{(k+1)}}{\partial \tilde{h}^{(k)}} \right) \sum_{l=k+1}^{T} \left(\frac{\partial \tilde{h}^{(l)}}{\partial \tilde{h}^{(k+1)}} \right) \delta_{\tilde{h}}^{(l)} \\ &= \delta_{\tilde{h}}^{(k)} + \left(\frac{\partial \tilde{h}^{(k+1)}}{\partial \tilde{h}^{(k)}} \right) \Delta_{\tilde{h}}^{(k+1)} \end{aligned} \tag{7.11}$$

Finally, use chain rule again to write

$$\delta_{\tilde{h}}^{(k)} = \frac{\partial h^{(k)}}{\partial \tilde{h}^{(k)}} \delta_h^{(k)}$$

■

Exercise 2.

i. First determine h_T by induction:

$$\begin{aligned} h^{(1)} &= wh^{(0)} + ux \\ h^{(2)} &= wh^{(1)} = w^2h^{(0)} + uwx \\ h^{(3)} &= wh^{(2)} = w^3h^{(0)} + uw^2x \\ &\vdots \\ h^{(t)} &= w^th^{(0)} + uw^{t-1}x = uw^{t-1}x \end{aligned}$$

since $h^{(0)} = 0$. Substituting into the loss yields

$$\mathcal{L} = \frac{1}{2}\left(\hat{y} - y\right)^2 = \frac{1}{2}\left(uw^{T-1}x - y\right)^2, \tag{7.12}$$

which has a unique minimiser at $\hat{y} = y$, so then

$$w^* = \left(\frac{y}{ux}\right)^{\frac{1}{T-1}}$$

when $T-1$ odd or $T-1$ even and $\dfrac{y}{ux} \geq 0$. If $u, x = 0$ or $T-1$ is even and $\dfrac{y}{ux} < 0$, then no real exact-fit solution exists (see also Lab Week 7 exercise 1).

ii. A standard BPTT implementation computes $\frac{d\mathcal{L}}{dw}$ by first performing a forward pass, followed by a backward pass through time.

$$\begin{aligned} \textbf{Forward pass:} \quad & \text{Set } h^{(0)} = 0, \text{For } t = 1, \ldots, T, \\ & h^{(t)} = wh^{(t-1)} + ux^{(t)}. \end{aligned}$$

To evaluate the Backward pass, evaluate all the necessary derivatives

$$\begin{aligned} \frac{\partial h^{(k)}}{\partial w} &= h^{(k-1)} \\ \dot{\phi}(a^{(j)} &= 1, \quad \forall\ j \\ \delta_h^{(l)} &= \begin{cases} \frac{d}{dh^{(l)}}\frac{1}{2}(h^{(l)} - y)^2 & l = T \\ 0, & l \neq T \end{cases} \end{aligned}$$

$$\begin{aligned} \textbf{Backward pass: } & \text{Set} \frac{d\mathcal{L}}{dw} = 0,\ \delta_h^{(T)} = \delta_a^{(T)} = h^{(T)} - y. \\ & \text{For } t = T, T-1, \ldots, 2, \\ & \quad \frac{d\mathcal{L}}{dw} \leftarrow \frac{d\mathcal{L}}{dw} + \delta_a^{(t)} h^{(t-1)}, \\ & \quad \delta_a^{(t-1)} \leftarrow w\,\delta_a^{(t)}. \end{aligned}$$

As an aside, note that using (**??**), and $h^{(1)} = 0$,

$$\begin{aligned}\frac{d\mathcal{L}}{dw} &= \delta_h^{(T)} \sum_{k=2}^{T} \left(\prod_{j=k+1}^{T} w \right) h^{(k-1)} \\ &= (h^{(T)} - y)(T-1)w^{T-2}ux \\ &= (T-1)(uw^{T-1}x^{(1)} - y)w^{T-2}ux\end{aligned}$$

This expression is identical to what we would get by directly differentiating (7.12) with respect to w, so we can be sure the expression is correct.

iii. From the closed-form expression obtained in part (ii),

$$\begin{aligned}\frac{d\mathcal{L}}{dw} &= (T-1)\big(uxw^{T-1} - y\big)uxw^{T-2} \\ &= (T-1)u^2(x)^2w^{2T-3} - (T-1)yuxw^{T-2}.\end{aligned}$$

Now consider the limit as $T \to \infty$ for fixed w.

- If $|w| < 1$, $w^{T-1} \to 0$ and also $Tw^{T-2} \to 0$, so

$$\lim_{T\to\infty} \frac{d\mathcal{L}}{dw} = 0.$$

  Hence the gradient vanishes for $|w| < 1$.
- $|w|^{T-1}$ and $|w|^{T-2}$ grow exponentially with T, so the gradient magnitude diverges,

$$\lim_{T\to\infty} \left| \frac{d\mathcal{L}}{dw} \right| = \infty$$

  Hence the gradient explodes for $|w| > 1$.
- If $|w| = 1$, the gradient does not vanish in general; for example, at $w = 1$ it typically grows linearly in T unless $ux = y$.

You can visualise this behaviour in Lab Week 7 exercise 1. This shows why gradient descent can become badly conditioned for long sequences, and why BPTT alone does not remove the problem. A typical remedy is gradient clipping for exploding gradients, and gated architectures such as LSTMs/GRUs for mitigating vanishing gradients.

Exercise 3.

i. Expanding the latent autoregressive state gives

$$\begin{aligned}s^{(1)} &= \beta_\star x^{(1)}, \\ s^{(2)} &= a_\star \beta_\star x^{(1)} + \beta_\star x^{(2)}, \\ s^{(3)} &= a_\star^2 \beta_\star x^{(1)} + a_\star \beta_\star x^{(2)} + \beta_\star x^{(3)}.\end{aligned}$$

Therefore, by induction,

$$s^{(T)} = \beta_\star \sum_{j=1}^{T} a_\star^{T-j} x^{(j)}.$$

Since $y^{(T)} = \psi(s^{(T)})$, we obtain

$$y^{(T)} = \psi\left(\beta_\star \sum_{j=1}^{T} a_\star^{T-j} x^{(j)}\right).$$

ii. The true data generating process applies the nonlinearity at the final time T only, whereas the RNN applies non-linear activation at every time t.

iii. Since the loss depends only on the final output, the only nonzero output error signal is at time T. Let

$$\delta_h^{(T)} := \frac{\partial \mathcal{L}}{\partial h^{(T)}} = h^{(T)} - y^{(T)},$$

and $\delta_h^{(l)} = 0$ for $l < T$. We then have

$$\frac{d\mathcal{L}}{dw} = \sum_{l=1}^{T}\left(\sum_{k=1}^{l}\frac{\partial h^{(k)}}{\partial w}\prod_{j=k+1}^{l}\left(w\dot{\phi}(a^{(j)})\right)\right)\delta_h^{(l)},$$
$$\frac{d\mathcal{L}}{du} = \sum_{l=1}^{T}\left(\sum_{k=1}^{l}\frac{\partial h^{(k)}}{\partial u}\prod_{j=k+1}^{l}\left(w\dot{\phi}(a^{(j)})\right)\right)\delta_h^{(l)},$$
$$\frac{d\mathcal{L}}{db} = \sum_{l=1}^{T}\left(\sum_{k=1}^{l}\frac{\partial h^{(k)}}{\partial b}\prod_{j=k+1}^{l}\left(w\dot{\phi}(a^{(j)})\right)\right)\delta_h^{(l)}.$$

Since

$$\frac{\partial h^{(k)}}{\partial w} = \dot{\phi}(a^{(k)})h^{(k-1)}, \qquad \frac{\partial h^{(k)}}{\partial u} = \dot{\phi}(a^{(k)})x^{(k)}, \qquad \frac{\partial h^{(k)}}{\partial b} = \dot{\phi}(a^{(k)}),$$

these become

$$\frac{d\mathcal{L}}{dw} = \delta_h^{(T)}\sum_{k=1}^{T}\left(\prod_{j=k+1}^{T} w\dot{\phi}(a^{(j)})\right)\dot{\phi}(a^{(k)})h^{(k-1)},$$
$$\frac{d\mathcal{L}}{du} = \delta_h^{(T)}\sum_{k=1}^{T}\left(\prod_{j=k+1}^{T} w\dot{\phi}(a^{(j)})\right)\dot{\phi}(a^{(k)})x^{(k)},$$
$$\frac{d\mathcal{L}}{db} = \delta_h^{(T)}\sum_{k=1}^{T}\left(\prod_{j=k+1}^{T} w\dot{\phi}(a^{(j)})\right)\dot{\phi}(a^{(k)}).$$

iv. If $|\dot{\phi}(a^{(j)})| \le \rho_\phi < 1$ for all j, then

$$\left|\frac{\partial h^{(T)}}{\partial h^{(k)}}\right| = \left|\prod_{j=k+1}^{T} w\,\dot{\phi}(a^{(j)})\right| \le (\rho_\phi |w|)^{T-k}$$

Therefore, looking at the loss gradients (e.g. with respect to u),

$$\left|\frac{d\mathcal{L}}{du}\right| \le \left|\delta_h^{(T)}\right| \sum_{k=1}^{T} \left|\prod_{j=k+1}^{T} w\dot{\phi}(a^{(j)})\right| \left|\dot{\phi}(a^{(k)})x^{(k)}\right| \le \left|\delta_h^{(T)}\right| \sum_{k=1}^{T} (\rho_\phi|w|)^{T-k} |\dot{\phi}(a^{(k)})||x^{(k)}|$$

This shows that the influence of an input $x^{(k)}$ on the final loss is multiplied by a factor that decays exponentially with the temporal distance $T-k$, provided $\rho_\phi|w| < 1$. Hence the gradient signal coming from early time steps becomes very small, so the RNN has difficulty learning long-range dependence.

By contrast, the true model satisfies

$$\frac{\partial y^{(T)}}{\partial x^{(k)}} = \dot{\psi}(s^{(T)})\,\beta_\star\, a_\star^{T-k}.$$

If $a_\star \approx 1$, then this dependence decays very slowly, so the data generating process genuinely retains long-range information. The problem is therefore not that the target function lacks long memory, but that BPTT transmits the learning signal through a product of many Jacobian factors, which can vanish much faster than the true sensitivity of $y^{(T)}$ to $x^{(k)}$.

Exercise 4.

i. Expanding the memory recurrence gives

$$\begin{aligned}
\tilde{h}^{(1)} &= g_i^{(1)} v x^{(1)}, \\
\tilde{h}^{(2)} &= g_f^{(2)} g_i^{(1)} v x^{(1)} + g_i^{(2)} v x^{(2)}, \\
\tilde{h}^{(3)} &= g_f^{(3)} g_f^{(2)} g_i^{(1)} v x^{(1)} + g_f^{(3)} g_i^{(2)} v x^{(2)} + g_i^{(3)} v x^{(3)}.
\end{aligned}$$

Therefore, by induction,

$$\tilde{h}^{(T)} = v \sum_{j=1}^{T} g_i^{(j)} \left(\prod_{r=j+1}^{T} g_f^{(r)}\right) x^{(j)}.$$

ii. The true latent state is

$$s^{(T)} = \beta_\star \sum_{j=1}^{T} a_\star^{T-j} x^{(j)}.$$

The LSTM memory state is

$$\tilde{h}^{(T)} = v \sum_{j=1}^{T} g_i^{(j)} \left(\prod_{r=j+1}^{T} g_f^{(r)} \right) x^{(j)}.$$

To match the true model for all input sequences, it is sufficient to match the coefficient of each $x^{(j)}$. That is, we require

$$v g_i^{(j)} \prod_{r=j+1}^{T} g_f^{(r)} = \beta_\star a_\star^{T-j}, \qquad j = 1, \ldots, T.$$

A simple analytic choice is

$$\begin{aligned} v^\star &= \beta_\star, \\ g_i^{(j)\star} &= 1, \qquad j = 1, \ldots, T, \\ g_f^{(r)\star} &= a_\star, \qquad r = 2, \ldots, T. \end{aligned}$$

Then

$$v^\star g_i^{(j)\star} \prod_{r=j+1}^{T} g_f^{(r)\star} = \beta_\star a_\star^{T-j}.$$

Hence

$$\tilde{h}^{(T)} = s^{(T)},$$

and therefore

$$\hat{y}^{(T)} = \psi(\tilde{h}^{(T)}) = \psi(s^{(T)}) = y^{(T)}.$$

This exact choice is possible when $0 \le a_\star \le 1$ and the gates are allowed to take values in the closed interval $[0, 1]$. If the gates are implemented as sigmoid outputs, then values exactly equal to 0 or 1 are not attained with finite pre-activations, but they can be approximated arbitrarily closely.

iii. Since the loss depends only on the final output, define the final error signal

$$\delta_{\tilde{h}}^{(T)} := \frac{\partial \mathcal{L}}{\partial \tilde{h}^{(T)}} = \left(\hat{y}^{(T)} - y^{(T)} \right) \dot{\psi}\left(\tilde{h}^{(T)} \right).$$

From

$$\tilde{h}^{(T)} = v \sum_{j=1}^{T} g_i^{(j)} \left(\prod_{r=j+1}^{T} g_f^{(r)} \right) x^{(j)},$$

we obtain

$$\frac{\partial \mathcal{L}}{\partial v} = \delta_{\tilde{h}}^{(T)} \sum_{j=1}^{T} g_i^{(j)} \left(\prod_{r=j+1}^{T} g_f^{(r)} \right) x^{(j)},$$

$$\frac{\partial \mathcal{L}}{\partial g_i^{(t)}} = \delta_{\tilde{h}}^{(T)} \, v \left(\prod_{r=t+1}^{T} g_f^{(r)} \right) x^{(t)},$$

$$\frac{\partial \mathcal{L}}{\partial g_f^{(t)}} = \delta_{\tilde{h}}^{(T)} \sum_{j=1}^{t-1} v \, g_i^{(j)} \left(\prod_{r=j+1}^{T} g_f^{(r)} \right) x^{(j)} = \delta_{\tilde{h}}^{(T)} \left(\prod_{r=t+1}^{T} g_f^{(r)} \right) \tilde{h}^{(t-1)}.$$

iv. The key point is that the internal memory state has a direct additive recurrence. In this simplified model,

$$\tilde{h}^{(t)} = g_f^{(t)} \tilde{h}^{(t-1)} + g_i^{(t)} v x^{(t)},$$

so the sensitivity of the final memory state to an earlier memory state is

$$\frac{\partial \tilde{h}^{(T)}}{\partial \tilde{h}^{(j)}} = \prod_{r=j+1}^{T} g_f^{(r)}.$$

Because each forget gate satisfies $0 \leq g_f^{(r)} \leq 1$, this product cannot explode, and it decays only when the forget gates stay significantly below 1.

v. The RNN propagates gradients through the factors

$$w \dot{\phi}(a^{(r)}),$$

which depend simultaneously on the recurrent weight and the activation derivative. Since $\dot{\phi}(a^{(r)})$ is often much smaller than 1, these products can decay rapidly over long time intervals.

By contrast, part (ii) showed that when the true system has long memory ($a_\star \approx 1$), the optimal LSTM parameters satisfy

$$g_f^{(r)} = a_\star \approx 1.$$

Hence the memory-path gradient

$$\frac{\partial \tilde{h}^{(T)}}{\partial \tilde{h}^{(k)}} = \prod_{r=k+1}^{T} g_f^{(r)}$$

remains close to 1, allowing gradients to propagate with much less attenuation. However, if during BPTT, the learned forget gates satisfy $g_f^{(r)} < 1$ over a long interval, this product still decays exponentially. Thus LSTMs mitigate, but do not completely eliminate, the vanishing-gradient problem.